Lab 2 - 3: Data Safety and Cost Management (Lifecycle & Versioning)

1. Introduction & Scenario

Your work as the data manager for "Project Starlight" has been a great success. The S3 buckets are organized, and the research team is thrilled. However, success brings two new challenges:

1. **Data Safety:** A team member accidentally uploaded a lower-quality image, overwriting a perfectly processed, high-resolution photo of a nebula. How can you protect against these kinds of mistakes?

2. **Cost Management:** The telescope generates massive amounts of raw sensor data daily. Most of this data is only needed for a few weeks, but it must be kept for long-term audits. How can you reduce storage costs without deleting this data?

In this lab, you will solve these problems by implementing two of S3's most powerful features: **Versioning** and **Lifecycle Policies**. We will follow our established learning pattern: first, you'll use the AWS Console to see how these features work visually, and then you'll implement the exact same configuration using the command line in CloudShell.

2. Learning Objectives

By the end of this lab, you will be able to:

- Enable and observe the behavior of S3 object versioning.
- Configure an S3 lifecycle policy to automatically change an object's storage class.
- Implement both of these features using the AWS Console and the AWS CloudShell.

3. Prerequisites

- An AWS account and access to the AWS Management Console and CloudShell.
- A sample image and a sample log file. You will create these directly in CloudShell.

Part A: The Visual Method (AWS Console)

Step 1: Setup Your New Bucket

Let's start with a fresh bucket for this experiment.

1. Navigate to the **S3 Console**.
2. Create a new bucket. Use the naming convention `starlight-archive-yourname-` followed by today's date (e.g., `starlight-archive-janedoe-20250917`).
3. Once the bucket is created, click its name and create two folders: `processed-images` and `raw-data`.
4. Navigate into the `processed-images` folder and upload the `andromeda.jpg` file from the previous labs.

Step 2: Enabling Versioning

Versioning is your safety net. When enabled, S3 will automatically keep a copy of every version of a file.

1. Go back to your bucket's main page.

2. Click on the **Properties** tab.
3. Scroll down to the **Bucket Versioning** section and click **Edit**.
4. Select the **Enable** option and click **Save changes**.

That's it! Versioning is now active for every object in this bucket.

Step 3: Observing Versioning in Action

1. Navigate back into your `processed-images` folder.
2. Click **Upload** and upload the *exact same* `andromeda.jpg` file again. It will look like it simply overwrote the original.
3. But now, at the top of the object list, you will see a toggle switch labeled **Show versions**. Click this to slide it to the "on" position.
4. You will now see **two** entries for `andromeda.jpg`. One is the current version (marked "Latest version"), and one is the older copy. S3 saved it for you automatically! You can download, restore, or delete specific versions from this view.
5. Turn the **Show versions** toggle back off for now.

Step 4: Creating a Lifecycle Policy

Now, let's address the cost issue. We'll tell S3 to automatically move our raw data to a cheaper storage tier after 30 days.

1. Go back to your bucket's main page and click the **Management** tab.
2. Under **Lifecycle rules**, click **Create lifecycle rule**.
3. Give the rule a name, like `ArchiveRawData`.
4. For **Rule scope**, choose "Limit the scope of this rule using one or more filters."
5. In the filter box, enter a prefix: `raw-data/`. This tells the rule to only apply to objects in that folder.
6. Under **Lifecycle rule actions**, check the box for "Transition current versions of objects between storage classes".
7. In the new section that appears, choose the storage class **Glacier Instant Retrieval**. For **Days after object creation**, enter `30`.
8. Scroll to the bottom and click **Create rule**.

You have now configured an automated cost-saving policy!

Part B: The Automation Method (AWS CloudShell)

Let's replicate that entire setup using only the command line.

Step 1: Setup Your New CLI Bucket

1. Launch **AWS CloudShell**.
2. Create the files we need for this lab:

```
mkdir archive_lab && cd archive_lab

wget https://cdn.esahubble.org/archives/images/publicationjpg/heic1502a.jpg -O andromeda.jpg

echo "SENSOR LOG DATA..." > sensor-data.log
```

3. Create a new, separate bucket for the command-line part.

```
# Remember to change yourname and the date!

BUCKET_NAME="starlight-archive-cli-yourname-20250917"

aws s3 mb s3://$BUCKET_NAME
```

4. Upload the initial files to their correct prefixes.

```
aws s3 cp andromeda.jpg s3://$BUCKET_NAME/processed-images/

aws s3 cp sensor-data.log s3://$BUCKET_NAME/raw-data/
```

Step 2: Enabling Versioning (CLI)

1. Use the `s3api put-bucket-versioning` command to enable versioning.

```
aws s3api put-bucket-versioning --bucket $BUCKET_NAME --versioning-configuration Status=Enabled
```

2. If the command is successful, there will be no output.

Step 3: Observing Versioning in Action (CLI)

1. Upload the `andromeda.jpg` file a second time to the exact same location.

```
aws s3 cp andromeda.jpg s3://$BUCKET_NAME/processed-images/
```

2. Now, list the object versions using the `s3api list-object-versions` command.

```
aws s3api list-object-versions --bucket $BUCKET_NAME --prefix processed-images/
```

3. Observe the JSON output. Under the `"Versions"` key, you will see two distinct objects, each with a unique `"VersionId"`. The most recent upload will have `"IsLatest": true`.

Step 4: Creating a Lifecycle Policy (CLI)

1. We need to define our rule in a JSON file. Use the `nano` text editor to create the file.

```
nano lifecycle.json
```

2. Paste the following JSON into the editor. This is the programmatic equivalent of the rule we created in the console.

```
{
  "Rules": [
    {
      "ID": "ArchiveRawDataCLI",
```

```
      "Status": "Enabled",
      "Filter": {
        "Prefix": "raw-data/"
      },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "GLACIER_IR"
        }
      ]
    }
  ]
}
```

3. Save and exit `nano`: press `Ctrl+X`, then `Y`, then `Enter`.
4. Finally, apply this policy to your bucket using the `s3api put-bucket-lifecycle-configuration` command.

   ```
   aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET_NAME --lifecycle-configuration file://lifecycle.json
   ```

5. Again, no output means success.

Deliverable

To receive completion marks for this lab, you must submit a screenshot of your CloudShell window after completing the tasks in Part B. Your screenshot should demonstrate that you successfully applied the lifecycle policy.

To capture the best proof, please run the following verification command and include its output in your screenshot:

aws s3api get-bucket-lifecycle-configuration --bucket $BUCKET_NAME

Your screenshot should show the JSON output that displays the "ArchiveRawDataCLI" rule you configured.

5. Cleanup

Use the CLI to delete both buckets and all their contents. Since the buckets have versions, a simple `rm` won't work on its own. You need to delete all the objects and their versions first, which you can do by emptying the bucket.

1. **Empty and delete both buckets** (run these commands for both bucket names you created):

   ```
   # First, get the name of your console bucket
   CONSOLE_BUCKET_NAME="starlight-archive-yourname-20250917"
   ```

```
# Now empty and delete both buckets

aws s3 rb s3://$CONSOLE_BUCKET_NAME --force

aws s3 rb s3://$BUCKET_NAME --force
```

The --force flag will delete all objects, and then the bucket itself, which is very convenient for cleanup.

But since you have enabled the versioning, this command removes the files, but not the previous versions. As a result, it won't remove the s3 bucket. To do that you need to first remove the versions using their version id. For this lab, simply remove them from the console. To see how to use CLI for this, read here: https://docs.aws.amazon.com/AmazonS3/latest/userguide/DeletingObjectVersions.html

If you are still interested to remove it using cli,

aws s3api delete-objects --bucket $BUCKET_NAME --delete "$(aws s3api list-object-versions --bucket $BUCKET_NAME --output=json --query='{Objects: Versions[].{Key:Key,VersionId:VersionId}}')"

However, you still need to delete the delete markers using their id or using console. You have now mastered two of the most critical features for managing S3 data at scale, using both the friendly console and the powerful CLI.